

Library Management System (C++ Project)

Project Title

Library Management System

1. Introduction

The Library Management System is a software application developed to manage books, members, and book transactions efficiently. This system helps librarians maintain records of books issued, returned, and searched.

The project is developed using Object-Oriented Programming concepts in C++.

2. Objectives

To manage book records efficiently.

To maintain member details.

To issue and return books.

To search books by title or author.

To apply OOP concepts like classes and objects.

3. Features

Add Books

Display Books

Search Books by Title

Search Books by Author

Add Members

Display Members

Issue Books

Return Books

Menu Driven Program

4. Tools & Technologies Used

Programming Language: C++

IDE: Code::Blocks / VS Code / Turbo C++

Concepts Used:

Classes & Objects

Arrays

Functions

Conditional Statements

Loops

5. Algorithm

Step 1:

Start the program.

Step 2:

Display menu options.

Step 3:

Accept user choice.

Step 4:

Perform selected operation:

Add book

Display books

Search book

Add member

Issue book

Return book

Step 5:

Repeat until user exits.

Step 6:

Stop the program.

6. C++ Source Code

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Book
```

```

{
public:
    int bookID;
    string title;
    string author;
    bool issued;

    void addBook()
    {
        cout << "Enter Book ID: ";
        cin >> bookID;
        cin.ignore();

        cout << "Enter Book Title: ";
        getline(cin, title);

        cout << "Enter Author Name: ";
        getline(cin, author);

        issued = false;
    }

    void displayBook()
    {
        cout << "Book ID : " << bookID << endl;
        cout << "Title   : " << title << endl;
        cout << "Author  : " << author << endl;

        if (issued)
            cout << "Status : Issued" << endl;
        else
            cout << "Status : Available" << endl;

        cout << "-----" << endl;
    }
};

```

```

class Member
{
public:
    int memberID;
    string memberName;

    void addMember()

```

```

{
    cout << "Enter Member ID: ";
    cin >> memberID;
    cin.ignore();

    cout << "Enter Member Name: ";
    getline(cin, memberName);
}

void displayMember()
{
    cout << "Member ID   : " << memberID << endl;
    cout << "Member Name : " << memberName << endl;
    cout << "-----" << endl;
}
};

int main()
{
    Book books[100];
    Member members[100];

    int totalBooks = 0;
    int totalMembers = 0;

    int choice;

    do
    {
        cout << "\n===== LIBRARY MANAGEMENT SYSTEM =====" << endl;
        cout << "1. Add Book" << endl;
        cout << "2. Display Books" << endl;
        cout << "3. Search Book by Title" << endl;
        cout << "4. Search Book by Author" << endl;
        cout << "5. Add Member" << endl;
        cout << "6. Display Members" << endl;
        cout << "7. Issue Book" << endl;
        cout << "8. Return Book" << endl;
        cout << "9. Exit" << endl;

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)

```

```

{
case 1:
    books[totalBooks].addBook();
    totalBooks++;
    cout << "Book Added Successfully!" << endl;
    break;

case 2:
    if (totalBooks == 0)
    {
        cout << "No books available." << endl;
    }
    else
    {
        for (int i = 0; i < totalBooks; i++)
        {
            books[i].displayBook();
        }
    }
    break;

case 3:
{
    string searchTitle;
    cin.ignore();
    cout << "Enter Book Title to Search: ";
    getline(cin, searchTitle);

    bool found = false;

    for (int i = 0; i < totalBooks; i++)
    {
        if (books[i].title == searchTitle)
        {
            books[i].displayBook();
            found = true;
        }
    }

    if (!found)
        cout << "Book Not Found!" << endl;

    break;
}

```

case 4:

```
{
    string searchAuthor;
    cin.ignore();
    cout << "Enter Author Name to Search: ";
    getline(cin, searchAuthor);

    bool found = false;

    for (int i = 0; i < totalBooks; i++)
    {
        if (books[i].author == searchAuthor)
        {
            books[i].displayBook();
            found = true;
        }
    }

    if (!found)
        cout << "Author Not Found!" << endl;

    break;
}
```

case 5:

```
members[totalMembers].addMember();
totalMembers++;
cout << "Member Added Successfully!" << endl;
break;
```

case 6:

```
if (totalMembers == 0)
{
    cout << "No members available." << endl;
}
else
{
    for (int i = 0; i < totalMembers; i++)
    {
        members[i].displayMember();
    }
}
break;
```

case 7:

```
{
    int issueID;
    cout << "Enter Book ID to Issue: ";
    cin >> issueID;

    bool found = false;

    for (int i = 0; i < totalBooks; i++)
    {
        if (books[i].bookID == issueID)
        {
            found = true;

            if (!books[i].issued)
            {
                books[i].issued = true;
                cout << "Book Issued Successfully!" << endl;
            }
            else
            {
                cout << "Book Already Issued!" << endl;
            }
        }
    }

    if (!found)
        cout << "Book ID Not Found!" << endl;

    break;
}
```

case 8:

```
{
    int returnID;
    cout << "Enter Book ID to Return: ";
    cin >> returnID;

    bool found = false;

    for (int i = 0; i < totalBooks; i++)
    {
        if (books[i].bookID == returnID)
```

```

        {
            found = true;

            if (books[i].issued)
            {
                books[i].issued = false;
                cout << "Book Returned Successfully!" << endl;
            }
            else
            {
                cout << "Book Was Not Issued!" << endl;
            }
        }
    }

    if (!found)
        cout << "Book ID Not Found!" << endl;

    break;
}

case 9:
    cout << "Exiting Program..." << endl;
    break;

default:
    cout << "Invalid Choice!" << endl;
}

} while (choice != 9);

return 0;
}

```

7. Sample Output

===== LIBRARY MANAGEMENT SYSTEM =====

1. Add Book
2. Display Books
3. Search Book by Title
4. Search Book by Author

5. Add Member
6. Display Members
7. Issue Book
8. Return Book
9. Exit

Enter your choice: 1

Enter Book ID: 101

Enter Book Title: C++ Programming

Enter Author Name: Bjarne Stroustrup

Book Added Successfully!

8. Advantages

Easy to manage books.

Reduces manual work.

Fast searching process.

Maintains issue and return records.

User-friendly menu system.

9. Limitations

Data is not stored permanently.

No database connectivity.

Limited to basic operations.

10. Future Enhancements

Add file handling.

Add database support.

Create GUI application.

Add login system.

Add fine calculation feature.

11. Conclusion

The Library Management System successfully manages books, members, and transactions. The project demonstrates the use of Object-Oriented Programming concepts in C++ and provides an efficient solution for basic library operations.

12. Viva Questions with Answers

Q1. What is a Library Management System?

A system used to manage books, members, and library transactions.

Q2. Which language is used in this project?

C++.

Q3. Which OOP concepts are used?

Classes and Objects.

Q4. What is the purpose of issue and return features?

To track borrowed books.

Q5. What are future improvements?

Database, GUI, login system, and file handling.

13. How to Run the Program

1. Open Code::Blocks or VS Code.

2. Create a new C++ file.

3. Copy and paste the code.

4. Save the file as library.cpp.

5. Compile and run the program.

14. Project Submission Details

Project Name: Library Management System Language Used: C++ Type: Console Based Application

Submitted By:

Roll No:

College Name:

—